GUIA PRÁTICO · PM

Quer entender IA por dentro, sem virar engenheiro?

Passo a passo do AI Engineering from Scratch, curso aberto com 542 lições, e um roteiro de por onde um PM pode começar.

O QUE É

Um curso que você roda, não só lê.

542 lições
organizadas em 20 fases, da preparação do ambiente até projetos finais

Grátis e aberto
licença MIT, com mais de 71 mil estrelas no GitHub

4 linguagens
Python, TypeScript, Rust e Julia

Em português
o README tem versão pt-BR; as lições estão em inglês

A proposta do README: implementar por dentro modelos, busca e agentes, testar, inspecionar as falhas e guardar o código e os resultados.

ESCOLHA A ROTA

Você não precisa abrir as 542.

SE VOCÊ QUER	COMECE EM
Fundamentos de modelos	Fase 0, Setup e Tooling
Sistemas com LLM	Fase 11, LLM Engineering, lição Prompt Engineering
Agentes e entrega	Fase 14, Agent Engineering, lição The Agent Loop
Usar agentes de código em repositórios reais	Caminho Agent-Assisted Engineering
Definir a construção certa antes de implementar	Caminho Product Judgment and Delivery

O README chama de rotas de aprendizado e manda escolher um objetivo só. Para PM, as duas últimas linhas são as que mais conversam com o dia a dia.

TRÊS JEITOS DE APRENDER

Escolha um e comece hoje.

1. No site

abre a lição em aiengineeringfromscratch.com. Sem instalar nada, sem clonar o repositório

2. Com tutor de IA

o seu agente de código vira o professor, com quiz e plano de estudo salvo

3. Código local

clona o repositório e roda os comandos de cada lição no seu computador

Para quem não programa todo dia, o site é a porta de entrada mais curta. O tutor é o mais guiado.

PASSO 1 · TUTOR DE IA

Instale o tutor no seu agente.

Confira o que precisa

```
node --version
npx --version
python3 --version
```

Node.js, `npx` e um agente de código que aceite skills. O `python3` é para os laboratórios.

Instale

```
npx skills add rohitg00/ai-engineering-from-scratch
```

O instalador pergunta qual agente e qual escopo. Não precisa clonar o repositório. Confira se o agente enxerga a pasta da skill, como `.claude/skills/`.

Comece o curso

```
/start-learning
```

Esse é o comando no Claude Code. No Codex, escreva `start-learning`.

PASSO 2 · O QUE O TUTOR FAZ

As skills de aprendizado.

SKILL	PARA QUE SERVE
`start-learning`	Faz um quiz de nivelamento de dez perguntas e salva um plano em `LEARNING.md`
`learn`	Ensina uma lição por sessão: conceito, matemática, código e quiz, com fila de revisão
`course-guide`	Você diz "onde aprendo atenção?" e ele aponta as lições exatas
`check-understanding 13`	Quiz de oito perguntas da fase 13, com as lições para revisar

No Claude Code, todas começam com barra: `/learn`, `/course-guide`.

PASSO 3 · CÓDIGO LOCAL

Rode a primeira lição.

1. Baixe o repositório

```
git clone https://github.com/rohitg00/
ai-engineering-from-scratch.git
cd ai-engineering-from-scratch
```

Todos os comandos das lições saem da raiz, a pasta que tem o `README.md` e a pasta `phases/`.

2. Confira o ambiente

```
python3 phases/00-setup-and-tooling/
01-dev-environment/code/verify.py --route beginner
```

O README diz que cada falha vem com o motivo e um comando para corrigir.

3. Rode a primeira de matemática

```
python3 phases/01-math-foundations/
01-linear-algebra-intuition/code/vectors.py
```

Sem dependências. Mostra que matriz vezes vetor é a conta dentro de uma camada de rede neural.

A FORMA DE UMA LIÇÃO

Toda lição tem a mesma estrutura.

```
phases/<fase>/<lição>/
├── code/      código que roda
├── docs/en.md  a explicação
└── outputs/   o que a lição entrega
```

Seis tempos, sempre na mesma ordem:

Motto	a ideia central em uma linha
Problem	a dor concreta
Concept	diagramas e intuição
Build It	você escreve a versão pequena, sem framework
Use It	a mesma coisa com a biblioteca de produção
Ship It	vira prompt, skill, agente ou servidor MCP

ROTEIRO SUGERIDO PARA PM

Por onde começar, nesta ordem.

1. **Fase 11, Prompt Engineering.** O vocabulário básico de como se conversa com um modelo.
2. **Fase 13, The Tool Interface e Function Calling.** Entender o modelo chamando uma ferramenta.
3. **Fase 11, Context Engineering.** Por que o contexto acaba e o que decide o que entra nele.
4. **Fase 11, RAG.** Como o produto busca informação antes de responder.
5. **Fase 14, The Agent Loop.** Como um agente decide, age e onde costuma quebrar.
6. **Fase 11, Evaluation & Testing.** Como medir se melhorou de verdade.

O CAMINHO QUE É DE PM

Product Judgment and Delivery.

São as lições 47 a 54 da fase 14, em ordem:

47	Definir o resultado antes de escolher a saída
48	Descobrir o fluxo que as pessoas realmente fazem
49	Mapear premissas e resolver a mais arriscada primeiro
50	Escolher a menor fatia que muda a decisão
51	Escrever especificações que preservam o julgamento
52	Desenhar métricas de sucesso antes do resultado existir
53	Escolher entre protótipo, piloto ou produção
54	Construir um ciclo de feedback com dono e aposentadoria

Do resultado, passando por evidência, risco e escopo, até métrica, liberação em etapas e dono do feedback.

EXEMPLO 1 · AVALIAÇÃO DE BUSCA

Quando a média sobe e uma consulta piora.

O cenário, como o README conta

No projeto Retrieval Evaluation Lab, uma versão nova melhora a métrica média NDCG, mas uma consulta passa a trazer a evidência mais relevante mais para baixo. Você monta uma comparação consulta por consulta que acusa a regressão e pode reprovar uma liberação.

O QUE VOCÊ LEVA COMO PM

A habilidade de desconfiar de média. Na hora de aprovar uma melhoria em busca ou RAG, você pede o resultado por consulta, não só o número geral. O README avisa que a métrica descreve os julgamentos usados, não prova que a resposta está certa.

Projeto 01, em Python. Fonte: README do repositório.

EXEMPLO 2 · AGENTE LENTO OU CARO

Onde foi o tempo e onde foram os tokens.

O cenário, como o README conta

No Agent Trace Debugger, um registro de execução continua levando 100 ms, mas o uso total de tokens sobe 200 e uma etapa passa a falhar. Você separa o trabalho filho que roda em paralelo do tempo do pai e gera uma linha do tempo em HTML e um relatório de regressão em JSON.

O QUE VOCÊ LEVA COMO PM

Saber ler o rastro de um agente. A conversa com o time deixa de ser "está lento" e vira "esta etapa falhou e esta consumiu mais". O README ressalta contar os tokens de cada etapa sem somar pai e filho duas vezes.

Projeto 02, em TypeScript (Node.js 22.18 ou mais novo). Fonte: README.

EXEMPLO 3 · LIMITE DE AÇÃO DO AGENTE

O que o agente pode escrever, e quem aprovou.

O cenário, como o README conta

No Tool Call Firewall, uma escrita muda depois da revisão, ou uma aprovação é reaproveitada. Você valida o pedido, confere o papel de quem chamou e o caminho, e só consome uma aprovação ligada àquele pedido e àquele conteúdo exatos.

O QUE VOCÊ LEVA COMO PM

Vocabulário para requisitos de segurança: papel, caminho, aprovação de uso único e recibo de auditoria. O README deixa claro que o projeto não oferece autorização persistente nem isolamento do sistema operacional.

Projeto 03, em Rust. Fonte: README.

CUIDADOS

Do jeito certo, sem se afogar.

1. Não tente fazer tudo. São 542 lições. O próprio README manda escolher um objetivo e ir até o fim dele.

2. Tempo: o README estima cerca de 306 horas para quem começa do zero e cerca de 60 horas só na fase de agentes, para quem já é engenheiro.

3. Se você não programa, comece pelo site e pelo tutor, e use o código local quando já estiver confortável com o terminal.

4. Custo de API: o README não informa valores. Na fase 0 há uma lição sobre chaves de API, e vale conferir se cada lição chama algum serviço antes de rodar.

5. Tudo é em inglês, com README em português. Peça ao seu agente para explicar a lição em português.

O PONTO

Você não vira engenheiro, mas passa a discutir custo, risco e prazo com base.

Entender o modelo chamando ferramenta, o contexto, a busca, o agente e a avaliação muda o nível da conversa com o time.

AI Engineering from Scratch, de Rohit Ghumare: licença MIT, mais de 71 mil estrelas. O link está no primeiro comentário.